Phân cụm dữ liệu

Trần Hồng Việt

Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội

Nội dung

1. Khái niệm phân cụm dữ liệu
2. Hiểu dữ liệu trước khi phân cụm
3. Phân cụm phân cấp (Hierarchical clustering)
4. Phân cụm phân vùng (Partitioning clustering)
5. Phân cụm theo phân phối (Distribution-based)
6. Phân cụm theo mật độ (Density-based)
7. Kiểm định và đánh giá kết quả phân cụm
8. Tài liệu tham khảo và Kết luận

Nội dung

1. **Khái niệm phân cụm dữ liệu**
2. Hiểu dữ liệu trước khi phân cụm
3. Phân cụm phân cấp (Hierarchical clustering)
4. Phân cụm phân vùng (Partitioning clustering)
5. Phân cụm theo phân phối (Distribution-based)
6. Phân cụm theo mật độ (Density-based)
7. Kiểm định và đánh giá kết quả phân cụm
8. Tài liệu tham khảo và Kết luận

Phân cụm dữ liệu là gì?

Định nghĩa theo *Data Mining: Concepts and Techniques*, J. Han et al. [1]

Phân tích cụm (hay phân cụm) là quá trình phân hoạch một tập hợp các đối tượng dữ liệu (hay quan sát) thành các tập con. Mỗi tập con là một cụm, sao cho các đối tượng trong cùng một cụm *tương tự* nhau, nhưng *khác biệt* so với các đối tượng trong các cụm khác.

Định nghĩa theo *Mining of Massive Datasets*, J. Leskovec et al. [3]

Phân cụm là quá trình xem xét một tập hợp các điểm và nhóm chúng thành các cụm theo một độ đo khoảng cách nào đó. Mục tiêu là các điểm trong cùng một cụm có khoảng cách nhỏ với nhau, còn các điểm thuộc các cụm khác nhau thì ở cách xa nhau.

Định nghĩa theo Wikipedia

Phân tích cụm hay phân cụm là nhiệm vụ nhóm một tập hợp các đối tượng sao cho các đối tượng trong cùng nhóm (cụm) *tương tự* nhau hơn so với các đối tượng thuộc các nhóm khác.

Phân cụm dữ liệu (tiếp theo)

- Phân cụm dữ liệu còn được gọi là **học không giám sát** (unsupervised learning) hay phân loại không giám sát.
  - Phân loại (học có giám sát) là học từ ví dụ có nhãn, còn phân cụm là học từ quan sát không có nhãn.
- Hai loại phân cụm chính:
  - **Phân cụm cứng (Hard clustering):** mỗi điểm dữ liệu chỉ thuộc đúng một cụm.
  - **Phân cụm mềm (Soft clustering):** mỗi điểm dữ liệu có thể thuộc một hoặc nhiều cụm.
- Một số đặc điểm quan trọng:
  - Số lượng cụm của một tập dữ liệu thường không biết trước hoặc không rõ ràng.
  - Có nhiều phương pháp phân cụm, mỗi phương pháp có nhiều kỹ thuật cụ thể.
  - Các phương pháp/kỹ thuật phân cụm khác nhau có thể cho kết quả khác nhau trên cùng một tập dữ liệu.

Bài toán phân cụm dữ liệu

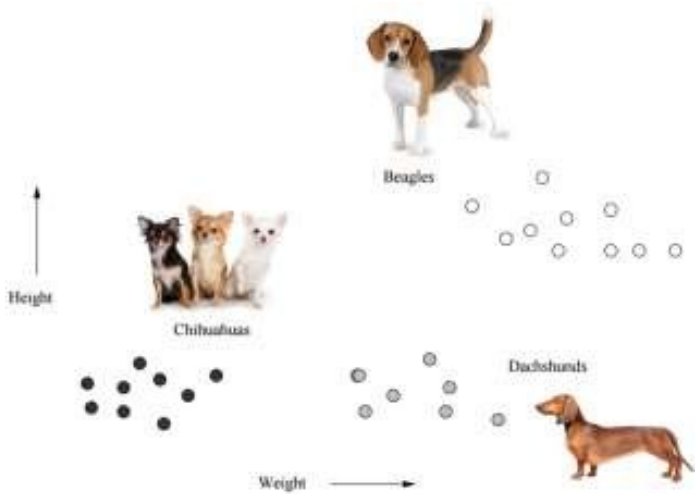


- Cho $X = (X_1, X_2, \ldots, X_d)$ là không gian d chiều, trong đó mỗi thuộc tính/biến X_j là số thực hoặc phân loại (categorical).
- Cho $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$ là tập mẫu (hay tập dữ liệu) gồm n điểm dữ liệu (còn gọi là thể hiện, quan sát, ví dụ, hoặc bộ dữ liệu) $x_i = (x_1, x_2, \ldots, x_d) \in X$.
- Bài toán phân cụm dữ liệu là dùng một thuật toán phân cụm A để gán các điểm trong $\mathcal{D}$ vào các cụm phù hợp nhất. Kết quả phân cụm là một tập k cụm:

$$C = \{C_1, C_2, \ldots, C_k\}$$

trong đó các điểm trong cùng một cụm *tương tự* nhau theo một nghĩa nào đó và *cách xa* các điểm thuộc các cụm khác.

Ví dụ về các điểm dữ liệu trong không gian 2 chiều



Nhận xét về quá trình và kết quả phân cụm

- Số cụm k được xác định theo hai cách: (1) k là tham số đầu vào của thuật toán, hoặc (2) k được tự động xác định bởi thuật toán.
- Thông thường, mỗi điểm dữ liệu chỉ thuộc một cụm (phân cụm cứng).
- Nếu điểm dữ liệu thuộc nhiều cụm (phân cụm mềm), mức độ thuộc cụm C_j của x_i được đặc trưng bởi trọng số w_{ij} (ví dụ: $w_{ij} \in [0, 1]$).
- Không phải tất cả các điểm trong $\mathcal{D}$ đều được gán vào cụm. Một số điểm có thể là **ngoại lệ** (outliers) hoặc **nhiễu** và bị loại khỏi các cụm.
- Kết quả phân cụm phụ thuộc vào thuật toán. Một số thuật toán dành cho phân cụm cứng, một số cho phân cụm mềm, một số xử lý được ngoại lệ và nhiễu.
- Việc gán điểm vào cụm được thực hiện **tự động**, do đó phân cụm hữu ích vì có thể dẫn đến phát hiện các nhóm *chưa được biết đến trước đây* trong dữ liệu.

Yêu cầu đối với phân cụm dữ liệu

- **Khả năng mở rộng (Scalability):** thuật toán phân cụm cần hoạt động hiệu quả và nhất quán với các tập dữ liệu nhỏ, vừa và lớn.
- **Xử lý nhiều loại thuộc tính:** có thể làm việc với các kiểu dữ liệu khác nhau như nhị phân, danh nghĩa (nominal), thứ tự (ordinal), số thực, hay hỗn hợp.
- **Phát hiện cụm hình dạng tùy ý:** các thuật toán dựa trên khoảng cách thường chỉ tìm được cụm hình cầu. Tuy nhiên, một cụm có thể có hình dạng bất kỳ, nên cần thuật toán có khả năng phát hiện cụm hình dạng tùy ý.
- **Tối thiểu hóa yêu cầu kiến thức miền:** phân cụm nên tự động hóa càng nhiều càng tốt, tránh phụ thuộc vào kiến thức miền (có thể gây thiên lệch).
- **Xử lý dữ liệu nhiễu:** hầu hết dữ liệu thực tế đều chứa ngoại lệ và/hoặc dữ liệu thiếu, không xác định, hoặc sai. Cần phương pháp phân cụm bền vững với nhiễu.

Yêu cầu đối với phân cụm dữ liệu (tiếp theo)

- **Phân cụm gia tăng và không nhạy cảm với thứ tự đầu vào:** trong nhiều ứng dụng, dữ liệu mới có thể đến bất kỳ lúc nào. Tốt hơn nếu thuật toán có thể xử lý dữ liệu mới theo cách gia tăng (incremental).
- **Phân cụm dữ liệu chiều cao:** tập dữ liệu có thể chứa rất nhiều chiều hoặc thuộc tính. Tìm cụm trong không gian chiều cao là thách thức lớn, đặc biệt vì dữ liệu có thể rất thưa và phân phối không đều.
- **Phân cụm dựa trên ràng buộc (Constraint-based):** ứng dụng thực tế có thể yêu cầu phân cụm dưới các ràng buộc, ví dụ: hai điểm nhất định không được nằm trong cùng một cụm hoặc ngược lại. Tích hợp ràng buộc vào thuật toán rất quan trọng trong một số lĩnh vực.
- **Tính giải thích và khả năng sử dụng:** người dùng muốn kết quả phân cụm có thể diễn giải được, dễ hiểu và hữu ích, gắn liền với các diễn giải ngữ nghĩa cụ thể.

Các phương pháp phân cụm

- **Phương pháp phân cấp (Hierarchical methods)** — còn gọi là phương pháp kết nối (connectivity methods)
  - Tạo ra phân hoạch phân cấp của dữ liệu dưới dạng cây cụm (dendrogram).
  - Có thể là *kết tụ* (agglomerative — từ dưới lên) hoặc *phân chia* (divisive — từ trên xuống).
  - Sử dụng các độ đo tương đồng khác nhau để tách hoặc gộp cụm.
  - Là phân cụm **cứng**; kết quả thường có dạng hình cầu.
- **Phương pháp phân vùng (Partitioning methods)** — còn gọi là phương pháp centroid
  - Dữ liệu được phân thành k cụm riêng biệt (k là tham số đầu vào).
  - Dựa trên cả centroid và khoảng cách.
  - Kỹ thuật phổ biến: k-means, k-medoids, k-medians, v.v.
  - Cũng là phân cụm **cứng**.
  - Phù hợp tìm cụm hình cầu trong tập dữ liệu nhỏ đến vừa.

Các phương pháp phân cụm (tiếp theo)

- **Phương pháp dựa trên phân phối (Distribution-based)** — còn gọi là mô hình xác suất (probabilistic models)
  - Giả định dữ liệu đến từ hỗn hợp các phân phối, ví dụ: phân phối chuẩn.
  - Phương pháp phổ biến: mô hình hỗn hợp Gaussian (GMMs) kết hợp thuật toán EM.
  - Là phân cụm **mềm**; các cụm có thể chồng chéo và có dạng hình elip.
  - Với cụm hình dạng tùy ý, phương pháp này có thể thất bại vì giả định phân phối thường không đúng.
- **Phương pháp dựa trên mật độ (Density-based methods)**
  - Ý tưởng: tiếp tục mở rộng cụm miễn là mật độ (số điểm dữ liệu) trong vùng lân cận vượt ngưỡng nào đó.
  - Phù hợp cho cụm có **hình dạng tùy ý**.
  - Có thể xử lý **nhiễu** và **ngoại lệ**.
  - Các thuật toán phổ biến nhất: DBSCAN và OPTICS.

Các phương pháp phân cụm (tiếp theo)

- **Phương pháp dựa trên lưới (Grid-based methods)**
  - Lượng hóa không gian đối tượng thành số hữu hạn các ô tạo thành cấu trúc lưới.
  - Tất cả các thao tác phân cụm được thực hiện trên cấu trúc lưới.
  - *Ưu điểm:* thời gian xử lý nhanh, phụ thuộc vào số lượng ô chứ không phải số điểm dữ liệu.
  - Hiệu quả cho phân cụm dữ liệu không gian, có thể kết hợp với phương pháp mật độ.
- **Các phương pháp khác**
  - *Phương pháp dựa trên đồ thị (Graph-based):* tìm cụm dựa trên khai thác đồ thị con dày đặc như clique hoặc quasi-clique.
  - *Mô hình không gian con (Subspace models):* cụm được mô hình hóa với cả thành viên cụm và các thuộc tính liên quan.
  - *Mô hình thần kinh (Neural models):* mạng thần kinh không giám sát phổ biến nhất là bản đồ tự tổ chức (Self-Organizing Map — SOM).

Bảng tóm tắt các phương pháp phân cụm

PP	Đặc điểm
Phân hoạch	- Cụm hình cầu, loại trừ - Dựa khoảng cách - Mean/medoid - Dữ liệu nhỏ–TB
Phân cấp	- Nhiều mức - Không sửa lỗi - Có linkage
Mật độ	- Hình dạng bất kỳ - Vùng mật độ - Ngưỡng lân cận - Bỏ ngoại lai
Lưới	- Lưới đa mức - Nhanh, phụ thuộc lưới

Table: Các phương pháp phân cụm

Thách thức trong phân cụm dữ liệu

- Phân cụm với **khối lượng dữ liệu lớn**.
- Phân cụm trong **không gian chiều cao** (high-dimensional space).
- Phân cụm với **dữ liệu chất lượng thấp** (ví dụ: nhiễu và giá trị thiếu).
- Phân cụm với **cấu trúc cụm phức tạp** (hình dạng, mật độ, chồng chéo, v.v.).
- Xác định đúng **giá trị tham số** phản ánh bản chất dữ liệu (ví dụ: số lượng cụm đúng, ngưỡng mật độ đúng, v.v.).
- **Kiểm định và đánh giá** kết quả phân cụm (không có nhãn để so sánh).

Phân cụm trong không gian chiều cao: lời nguyền chiều

- Trong không gian chiều rất cao, **hầu hết các cặp** điểm dữ liệu đều **rất xa nhau**. Cho hai điểm x và y trong $\mathcal{R}^d$, khoảng cách Euclidean là:

$$r(\mathrm{x}, \mathrm{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

- Giả sử tất cả điểm nằm trong **hình lập phương đơn vị** (mọi chiều trong $[0, 1]$). Cận dưới của $r(\mathrm{x}, \mathrm{y})$ là 1 (tồn tại ít nhất một chiều hai điểm rất xa nhau); cận trên là $\sqrt{d}$. Trên thực tế, hầu hết các cặp có khoảng cách trung bình:

$$\frac{1 + \sqrt{d}}{2}$$

- Khi d rất lớn, khoảng cách giữa các điểm dữ liệu trở nên **rất lớn** — mọi điểm đều "xa nhau như nhau", gây khó khăn nghiêm trọng cho phân cụm.

Phân cụm trong không gian chiều cao: lời nguyền chiều (tiếp theo)

- Trong không gian chiều rất cao, **hai vector tùy ý** gần như **trực giao với nhau**. Xét độ tương đồng cosine:

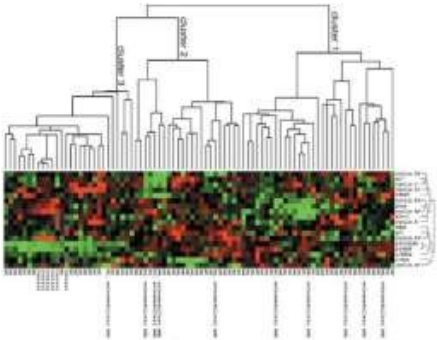
$$\cos(\mathrm{x}, \mathrm{y}) = \frac{\sum_{i=1}^{d} x_i\, y_i}{\sqrt{\sum_{i=1}^{d} x_i^2}\ \sqrt{\sum_{i=1}^{d} y_i^2}}$$

- Khi d rất lớn, tử số **nhỏ hơn rất nhiều** so với mẫu số, do đó cosine giữa hai vector rất **gần 0** — hai vector gần như trực giao.
- Nếu hầu hết các cặp điểm dữ liệu đều trực giao với nhau, việc phân cụm trở nên **rất khó khăn** và kết quả thường rất kém.
- Một giải pháp phổ biến là **giảm chiều dữ liệu** bằng các kỹ thuật như PCA, SVD, phân tích chủ đề (topic analysis), hoặc word embeddings (với dữ liệu văn bản).

Ứng dụng của phân cụm dữ liệu



[nguồn: bluffy.eecs.berkeley.edu]



[nguồn: www.nature.com]

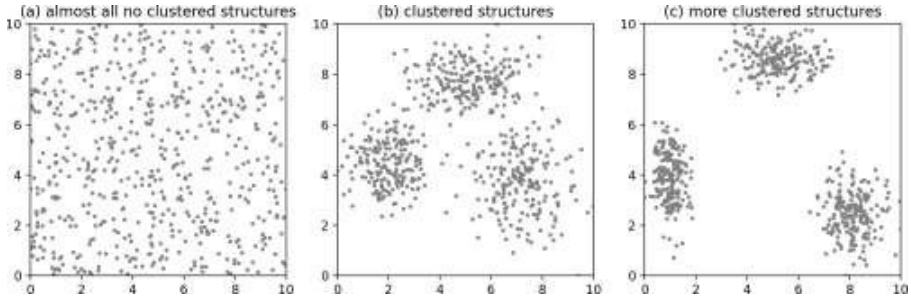
Ứng dụng của phân cụm dữ liệu (2)

- **Phân khúc khách hàng:** viễn thông, bán lẻ, marketing, tài chính – ngân hàng, v.v.
- **Phân cụm văn bản:** tin tức, email, dữ liệu chăm sóc khách hàng, gợi ý thẻ (tag suggestion), v.v.
- **Xử lý ảnh:** phân đoạn đối tượng (image segmentation), nhận dạng ảnh, v.v.
- **Dữ liệu sinh học:** phân cụm bệnh nhân, hồ sơ sức khỏe, biểu hiện gen (gene expression), v.v.
- **Mạng xã hội và đồ thị:** tìm người dùng tương đồng, phát hiện cộng đồng con (community detection), v.v.
- **Hệ thống gợi ý:** phân cụm người mua và sản phẩm (bán lẻ, thương mại điện tử), v.v.
- **Phát hiện bất thường:** nhận diện gian lận hoặc hành vi tội phạm, v.v.
- Phân cụm có thể là **bước tiền xử lý** cho các tác vụ phân tích và khai phá dữ liệu tiếp theo. Ứng dụng hiện diện ở khắp nơi trong phân tích dữ liệu.

Nội dung

1. Khái niệm phân cụm dữ liệu
2. **Hiểu dữ liệu trước khi phân cụm**
3. Phân cụm phân cấp (Hierarchical clustering)
4. Phân cụm phân vùng (Partitioning clustering)
5. Phân cụm theo phân phối (Distribution-based)
6. Phân cụm theo mật độ (Density-based)
7. Kiểm định và đánh giá kết quả phân cụm
8. Tài liệu tham khảo và Tóm tắt

Hiểu phân phối dữ liệu



- Dữ liệu có cấu trúc cụm không? Dữ liệu có thể phân cụm được không (**clusterability**)?
- Làm thế nào để đánh giá phân phối dữ liệu một cách **toán học** và **tự động**?

Các phương pháp xác định xu hướng phân cụm

- Histogram không gian (Spatial histogram / Cell-based histogram)
- Entropy theo ô (Cell-based entropy)
- Phân phối khoảng cách (Distance distribution)
- Thống kê Hopkins (Hopkins statistic)

Xác nhận xu hướng phân cụm bằng Histogram không gian

- Cách tiếp cận đơn giản: so sánh histogram không gian d chiều của tập dữ liệu D với histogram từ các mẫu được tạo ngẫu nhiên trong cùng không gian dữ liệu.
- Cho $X_1, X_2, \ldots, X_d$ là d chiều. Với b là số bin cho mỗi chiều, chia mỗi chiều X_j thành b bin có độ rộng đều, đếm số điểm nằm trong mỗi ô d chiều (b^d ô).
- Từ các d-histogram, ta thu được **hàm khối xác suất thực nghiệm khớp** (EPMF) cho tập D:

$$f(\mathrm{i}) = P(\mathrm{x}_j \in \text{cell i}) = \frac{|\{\mathrm{x}_j \in \text{cell i}\}|}{n} \tag{3}$$

trong đó $\mathrm{i} = (i_1, i_2, \ldots, i_d)$ là chỉ số ô, i_j là chỉ số bin theo chiều X_j; $n = |D|$.

Xác nhận xu hướng phân cụm bằng Histogram không gian (tiếp)

- Tạo t mẫu ngẫu nhiên, mỗi mẫu gồm n điểm trong cùng không gian d chiều với D. Với mỗi chiều X_j, tạo giá trị phân phối đều trong khoảng $[\min(X_j), \max(X_j)]$. Ký hiệu mẫu thứ j là R_j.
- Tính EPMF $g_j(\mathrm{i})$ cho từng R_j, $1 \leq j \leq t$.
- Đo mức độ khác biệt giữa f và g_j bằng **độ phân kỳ Kullback-Leibler (KL)**:

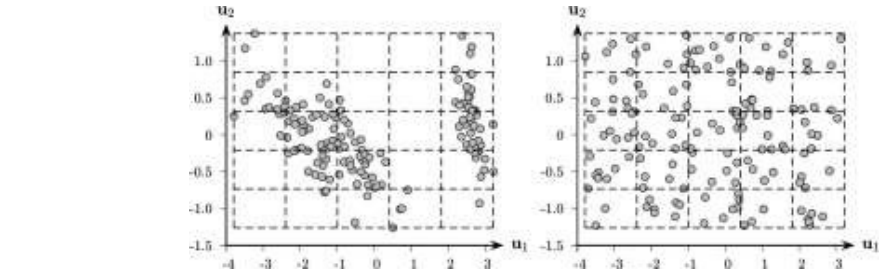
$$KL(f \| g_j) = \sum_{\mathrm{i}} f(\mathrm{i}) \log \left(\frac{f(\mathrm{i})}{g_j(\mathrm{i})} \right) \tag{4}$$

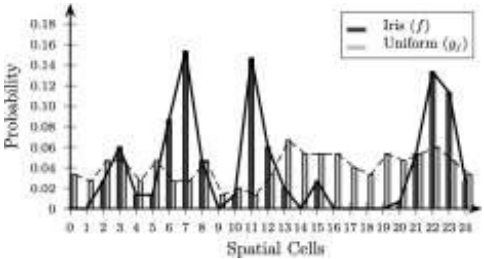
- Độ phân kỳ KL bằng 0 khi và chỉ khi f và g_j là cùng phân phối.
- Tính **kỳ vọng** và **phương sai** của $KL(f \| g_j)$ cho $j = 1 \ldots t$.

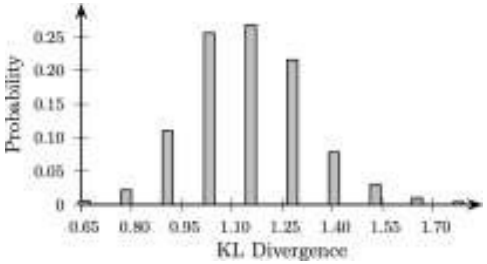
Ví dụ về Histogram không gian

- **Hạn chế chính:** khi chiều tăng, số ô (b^d) tăng theo hàm mũ; với kích thước mẫu cố định n, hầu hết ô sẽ trống, khó ước lượng độ phân kỳ. Phương pháp cũng nhạy cảm với tham số b.
- Ví dụ: tập dữ liệu Iris thành phần chính với $n = 150$, $d = 2$; dùng $b = 5$ bin mỗi chiều (25 ô không gian).
- Nó cũng hiển thị EPMF cho một trong các tập dữ liệu được tạo một cách ngẫu nhiên đồng đều trong cùng không gian dữ liệu. Cả hai EPMF đều được tính toán sử dụng b = 5 ô trong mỗi chiều, với tổng cộng 25 ô không gian.
- Với $t = 500$, tính toán độ lệch KL (Kullback-Leibler) từ f tới g_j cho mỗi $1 \leq j \leq t$ (sử dụng logarit cơ số 2).
- Kết quả thu được: trung bình $\mu_{KL} = 1.17$ và độ lệch chuẩn $\sigma_{KL} = 0.18$.
- $\Rightarrow$ Dữ liệu Iris **khác xa** dữ liệu ngẫu nhiên $\Rightarrow$ **có thể phân cụm được**.

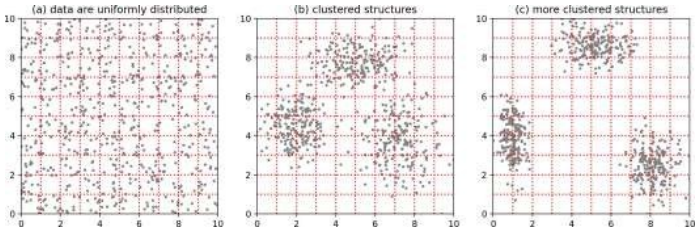
Ví dụ về Histogram không gian (tiếp)







Xác nhận xu hướng phân cụm bằng Entropy theo ô



- Không gian dữ liệu được chia thành lưới $k \times k$ ô. Ví dụ: $k = 10$, tổng số ô $m = 100$.
- Đếm số điểm dữ liệu trong mỗi ô cho ba trường hợp (a), (b) và (c).

Xác nhận xu hướng phân cụm bằng Entropy theo ô (tiếp)

```
[ 8 12  7 11  5  3  8  9  9  7] [ 0  0  1  4 10  6  0  0  0  0] [ 0  2 12 34 38 13  2  0  0  0]
[11  2  4  3  7  5  6  4  6  7] [ 0  0  2 23 27 22  4  0  0  0] [ 1  2 23 32 38 11  0  0  0  0]
[ 7  4  8  7  8  3  9  4  6  7] [ 0  0  5 22 25 25  5  2  4  0] [ 0  0  0  0  0  0  0  0  1  0]
[ 6  6  4  7  7  9  8  7  6  6] [ 0  0  0  7  9  3  6 12 10  1] [ 0  0  0  0  0  0  0  2  8  8]
[ 7  6  6  8  4  4  9  6  5  8] [ 0  0  1  1  3  1  8 29 19  2] [ 0  0  0  0  0  0  0 11 36 18]
[ 5  5  6  7  8  8  5  7  4  4] [ 0  1  3  5  6  2  5 34 14  3] [ 0  0  0  0  0  0  0 10 50 17]
[ 7  9  7  6  4  3  7  2  7  7] [ 1  3 14 23 13 11  7 19 14  2] [ 0  1  3  2  0  0  0  4 21  8]
[ 6  3  6  3  2  4 11  4  4  5] [ 0  7 12 23 21  8  2  3  5  2] [ 2 19 35 15  5  0  0  0  5  1]
[ 5  4  3  5  5  5  7  8  7  4] [ 0  5  7  5  6  3  2  0  0  0] [ 5 29 40 25  3  0  0  0  0  0]
[ 4  7  6  6  6  4  2  5  8  7] [ 1  0  3  3  2  1  0  0  0  0] [ 1  3  7  5  0  0  0  0  0  0]
```

- Tính **entropy** của phân phối điểm trên các ô, H:

$$H = -\sum_{i=1}^{m} p_i \log_2(p_i) \tag{5}$$

trong đó $p_i = c_i/n$, c_i là số điểm trong ô thứ i, n là tổng số điểm.

- Với $m = 100$ ô: giá trị entropy cực đại là $\log_2 100 = 6.6439$. Entropy có thể chuẩn hóa về $[0, 1]$ bằng $H/\log_2 m$.

Kết quả Entropy theo ô

- **Trường hợp (a)** — dữ liệu phân phối đều:
  Entropy $= 6.5539$; Entropy chuẩn hóa $= 0.9864 \approx 1.0$
- **Trường hợp (b)** — có cấu trúc cụm:
  Entropy $= 5.5318$; Entropy chuẩn hóa $= 0.8326$
- **Trường hợp (c)** — cấu trúc cụm rõ hơn:
  Entropy $= 4.8118$; Entropy chuẩn hóa $= 0.7242$
- **Entropy càng nhỏ**, dữ liệu **càng có cụm**. Entropy $= 0$ khi tất cả điểm rơi vào một ô.
- Phương pháp này phụ thuộc vào cách chia không gian dữ liệu thành các ô (tổng số ô m).

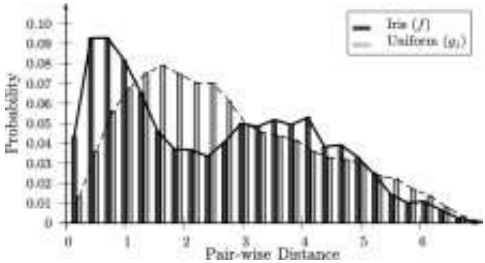
Xác nhận xu hướng phân cụm bằng Phân phối khoảng cách

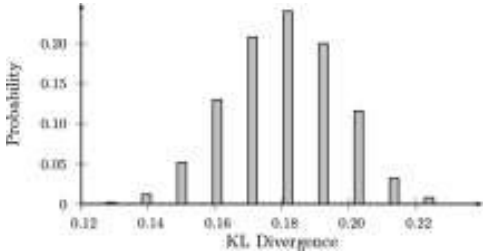
- Thay vì ước lượng mật độ, so sánh khoảng cách từng cặp điểm của D với các mẫu ngẫu nhiên R_j từ phân phối đều.
- Tính ma trận kề $W = \{w_{pq}\}_{p,q=1..n}$ bằng một phép đo khoảng cách.
- Xây dựng EPMF từ ma trận W bằng cách chia khoảng cách thành b bin:

$$f(i) = P(w_{pq} \in \text{bin } i \mid \mathrm{x}_p, \mathrm{x}_q \in D,\ p > q) = \frac{|\{w_{pq} \in \text{bin } i\}|}{n(n-1)/2} \tag{6}$$

- Tương tự, đối với mỗi mẫu R_j (j = 1..t) (phân bố đều), chúng ta có thể xác định EPMF cho các khoảng cách theo cặp, ký hiệu là g_j.
- Cuối cùng, tính toán sự khác biệt KL giữa f và g_j (với j = 1..t). Và tính toán kỳ vọng và phương sai của các giá trị khác biệt KL.

Ví dụ về Phân phối khoảng cách





- $b = 25$ bin; $t = 500$ mẫu. Trung bình $\mu_{KL} = 0.18$, $\sigma_{KL} = 0.017$.
- Mặc dù tập Iris có xu hướng phân cụm tốt, KL khá nhỏ.
- Mặc dù tập dữ liệu Iris có xu hướng phân cụm tốt, nhưng độ lệch KL không lớn lắm. Chúng tôi kết luận rằng, ít nhất đối với tập dữ liệu Iris, phân bố khoảng cách không phân biệt rõ ràng như phương pháp histogram không gian trong phân tích khả năng phân cụm.

Xác nhận xu hướng phân cụm bằng Thống kê Hopkins

- Cho $D = \{x_1, x_2, \ldots, x_n\}$ là tập n điểm trong $\mathbb{R}^m$.
- Chọn ngẫu nhiên h ($h < n$) điểm $\{x^1, \ldots, x^h\}$ từ D. Với mỗi x^i, tìm khoảng cách đến điểm gần nhất trong D:

$$a_i = \min_{u \in D} d(x^i, u) \tag{7}$$

- Tạo ngẫu nhiên h điểm giả $\{y^1, \ldots, y^h\}$ trong $\mathbb{R}^m$ theo phân phối đều (cùng khoảng giá trị với D). Với mỗi y^i, tìm khoảng cách đến điểm gần nhất trong D:

$$b_i = \min_{v \in D} d(y^i, v) \tag{8}$$

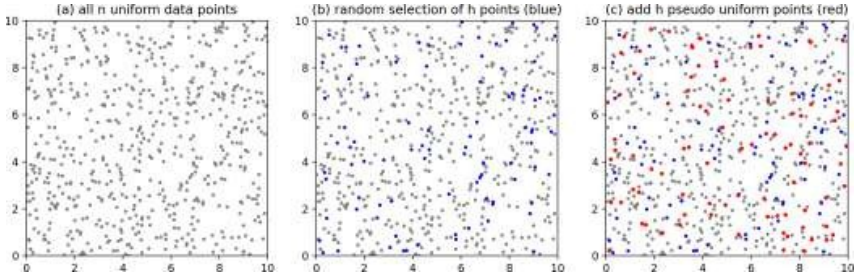
Thống kê Hopkins (tiếp)

- Thống kê Hopkins H được tính bằng:

$$H = \frac{\sum_{i=1}^{h} b_i}{\sum_{i=1}^{h} b_i + \sum_{i=1}^{h} a_i} \tag{9}$$

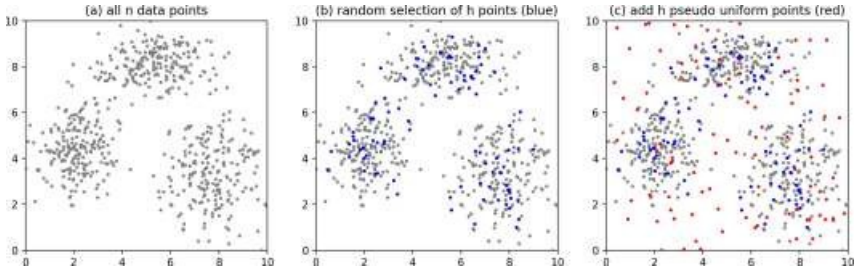
- Nếu dữ liệu trong D phân phối đều hoặc gần đều, $H \approx 0.5$.
- Nếu H gần 1.0, D có **cấu trúc cụm** (xa phân phối đều).

Ví dụ Thống kê Hopkins – Dữ liệu phân phối đều



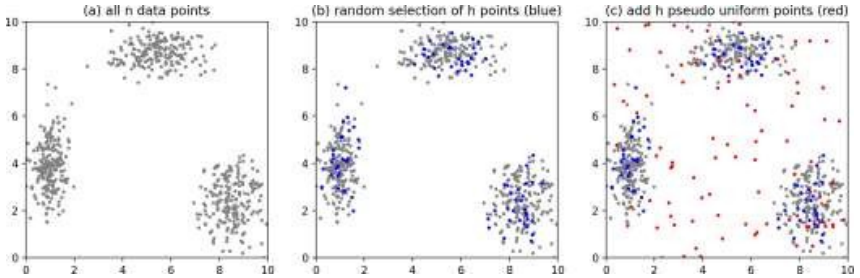
- D: $n = 600$ điểm phân phối đều, $h = 90$.
- $\sum a_i = 18.4981; \quad \sum b_i = 19.9432.$
- $H = \dfrac{19.9432}{19.9432 + 18.4981} = 0.5188 \approx 0.5$

Ví dụ Thống kê Hopkins – Cụm phân phối chuẩn



- D: $n = 600$ điểm, $h = 90$.
- $\sum_{i=1}^{h} a_i = 13.2464$; $\sum_{i=1}^{h} b_i = 45.1340$.
- $H = \dfrac{45.1340}{45.1340 + 13.2464} = 0.7731$

Ví dụ Thống kê Hopkins – Cụm phân phối chuẩn (tiếp)



- D: $n = 600$ điểm, $h = 90$.
- $\sum_{i=1}^{h} a_i = 9.3838$; $\sum_{i=1}^{h} b_i = 81.5614$.
- $H = \dfrac{81.5614}{81.5614 + 9.3838} = 0.8968$

Nội dung

1. Khái niệm phân cụm dữ liệu
2. Hiểu dữ liệu trước khi phân cụm
3. **Phân cụm phân cấp (Hierarchical clustering)**
4. Phân cụm phân vùng (Partitioning clustering)
5. Phân cụm theo phân phối (Distribution-based)
6. Phân cụm theo mật độ (Density-based)
7. Kiểm định và đánh giá kết quả phân cụm
8. Tài liệu tham khảo và Tóm tắt

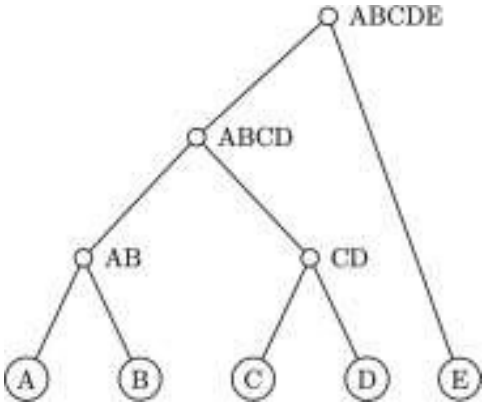
Phân cụm phân cấp (Hierarchical Clustering)

- Cho tập dữ liệu D bao gồm n điểm dữ liệu trong không gian d-chiều, mục tiêu của **phân cụm phân cấp** là tạo ra một chuỗi các phân vùng lồng nhau, có thể được trực quan hóa một cách thuận tiện thông qua một cây hoặc hệ thống phân cấp của các cụm, còn được gọi là **sơ đồ phân cụm**.
- Ở một mức trung gian nào đó, chúng ta có thể tìm thấy các cụm có ý nghĩa. Nếu người dùng cung cấp k, số lượng cụm mong muốn, chúng ta có thể chọn mức mà tại đó có k cụm.
- Hai hướng tiếp cận chính:
  - **Agglomerative** (bottom-up — từ dưới lên)
  - **Divisive** (top-down — từ trên xuống)

Phân cụm phân cấp (tiếp)

- Cho $D = \{x_1, x_2, \ldots, x_n\}$, trong đó $x_i \in \mathbb{R}^d$, một phân cụm $C = \{C_1, C_2, \ldots, C_k\}$ là một phân hoạch của D, tức là mỗi cụm là một tập các điểm dữ liệu $C_i \subseteq D$, sao cho các cụm đôi một rời nhau $C_i \cap C_j = \emptyset$ (với mọi $i \neq j$), và $\bigcup C_i = D$.
- Một phân cụm $A = \{A_1, A_2, \ldots, A_r\}$ được gọi là lồng trong một phân cụm khác $B = \{B_1, B_2, \ldots, B_s\}$ khi và chỉ khi $r > s$, và với mỗi cụm $A_i \in A$, tồn tại một cụm $B_j \in B$ sao cho $A_i \subseteq B_j$.
- Phân cụm phân cấp tạo ra một dãy gồm m phân hoạch lồng nhau $C_1, C_2, \ldots, C_m$, bắt đầu từ phân cụm tầm thường $C_1 = \{\{x_1\}, \{x_2\}, \ldots, \{x_n\}\}$ (mỗi điểm là một cụm riêng), đến phân cụm tầm thường còn lại $C_m = \{\{x_1, x_2, \ldots, x_n\}\}$ (tất cả điểm trong một cụm).
- Nói chung, phân cụm C_{t-1} được lồng trong phân cụm C_t.
- **Cây phân cấp (dendrogram)** là một cây nhị phân có gốc, biểu diễn cấu trúc lồng nhau này, với các cạnh nối giữa cụm $C_i \in C_{t-1}$ và cụm $C_j \in C_t$ nếu C_i được lồng trong C_j, tức là nếu $C_i \subset C_j$.

Dendrogram và các giải pháp phân cụm lồng nhau



clustering	clusters
$\mathcal{C}_1$	$\{A\},\{B\},\{C\},\{D\},\{E\}$
$\mathcal{C}_2$	$\{AB\},\{C\},\{D\},\{E\}$
$\mathcal{C}_3$	$\{AB\},\{CD\},\{E\}$
$\mathcal{C}_4$	$\{ABCD\},\{E\}$
$\mathcal{C}_5$	$\{ABCDE\}$

- Hình bên trái: dendrogram.
- Bảng bên phải: 5 mức phân cụm lồng nhau tương ứng.

Phân cụm phân cấp kết tụ (Agglomerative)

- Trong phân cụm phân cấp kết tụ, ta bắt đầu với mỗi trong n điểm dữ liệu là một cụm riêng biệt.
- Ta lặp lại việc gộp hai cụm gần nhau nhất cho đến khi tất cả các điểm thuộc về cùng một cụm (xem giả mã ở slide tiếp theo).
- Với tập cụm $C = \{C_1, C_2, \ldots, C_m\}$, ta tìm cặp cụm gần nhau nhất C_i và C_j, rồi gộp chúng thành cụm mới:

$$C_{ij} = C_i \cup C_j$$

- Sau đó cập nhật tập cụm bằng cách loại bỏ C_i và C_j, và thêm C_{ij}:

$$C = C \setminus \{C_i, C_j\} \cup \{C_{ij}\}$$

- Quá trình này lặp lại cho đến khi C chỉ còn một cụm. Nếu cần, ta có thể dừng khi còn đúng k cụm.

Phân cụm phân cấp kết tụ (Agglomerative) (tiếp)

AGGLOMERATIVE-CLUSTERING (D, k):

1. Mỗi điểm là một cụm riêng biệt: $\mathcal{C} \leftarrow \{C_i = \{x_i\} \mid x_i \in D\}$
2. $\Delta \leftarrow \{\delta(x_i, x_j) : x_i, x_j \in D\}$ // Tính ma trận khoảng cách
3. **repeat**
4. Tìm cặp cụm gần nhất $C_i, C_j \in \mathcal{C}$ [cite: 690]
5. $C_{ij} \leftarrow C_i \cup C_j$ // Gộp các cụm
6. $\mathcal{C} \leftarrow \mathcal{C} \setminus \{\{C_i\} \cup \{C_j\}\} \cup \{C_{ij}\}$ // Cập nhật
7. Cập nhật ma trận khoảng cách Δ để phản ánh phân cụm mới
8. **until** $|\mathcal{C}| = k$

Khoảng cách giữa các cụm

- Bước chính trong thuật toán là xác định cặp cụm gần nhất[cite: 698].
- Khoảng cách cụm-cụm dựa trên khoảng cách giữa hai điểm, thường được tính bằng khoảng cách Euclidean (L_2-norm):

$$\delta(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

- Có nhiều cách để đo lường độ gần gũi giữa hai cụm: liên kết đơn (single link), liên kết đầy đủ (complete link), liên kết trung bình (average link), liên kết trọng tâm (centroid link), bán kính (radius), và đường kính (diameter).

Khoảng cách giữa các cụm (tiếp theo)

- **Liên kết đơn (Single link):** Cho hai cụm C_i và C_j, khoảng cách giữa chúng, ký hiệu là $\delta(C_i, C_j)$, được định nghĩa là khoảng cách nhỏ nhất giữa một điểm trong C_i và một điểm trong C_j:

$$\delta(C_i, C_j) = \min\{\delta(x, y) \mid x \in C_i, y \in C_j\}$$

  Gộp bất kỳ hai cụm nào có khoảng cách liên kết đơn nhỏ nhất ở mỗi lần lặp.
- **Liên kết đầy đủ (Complete link):** Khoảng cách giữa hai cụm được định nghĩa là khoảng cách lớn nhất giữa một điểm trong C_i and một điểm trong C_j:

$$\delta(C_i, C_j) = \max\{\delta(x, y) \mid x \in C_i, y \in C_j\}$$

  Gộp bất kỳ hai cụm nào có khoảng cách liên kết đầy đủ nhỏ nhất tại mỗi vòng lặp.

Khoảng cách giữa các cụm (tiếp theo)

- **Liên kết trung bình (Average link):**

$$\delta(C_i, C_j) = \frac{\sum_{x \in C_i} \sum_{y \in C_j} \delta(x, y)}{n_i n_j}$$

với $n_i = |C_i|, n_j = |C_j|$. Gộp hai cụm có khoảng cách nhỏ nhất.

- **Liên kết trọng tâm (Centroid link):**

$$\delta(C_i, C_j) = \delta(\mu_i, \mu_j), \quad \mu_i = \frac{\sum_{x \in C_i} x}{n_i}, \ \mu_j = \frac{\sum_{y \in C_j} y}{n_j}$$

Gộp hai cụm có khoảng cách giữa các centroid nhỏ nhất.

Khoảng cách giữa các cụm (tiếp theo)

- **Bán kính (Radius):** Bán kính của một cụm là khoảng cách từ trọng tâm (giá trị trung bình) μ đến điểm xa nhất trong cụm đó:

$$r(C) = \max\{\delta(\mu_C, x) \mid x \in C\} \quad (15)$$

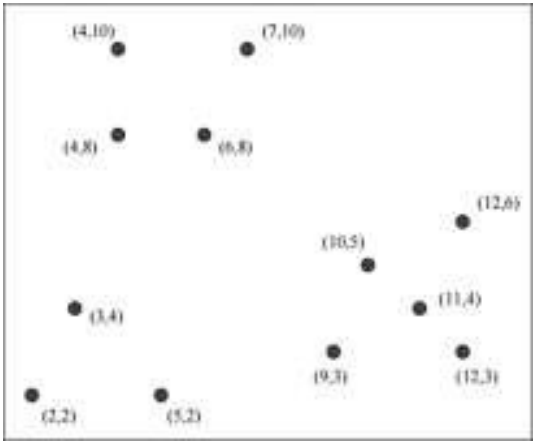
  Gộp bất kỳ hai cụm nào mà sau khi gộp tạo thành một cụm mới có bán kính nhỏ nhất tại mỗi lần lặp.

- **Đường kính (Diameter):** Đường kính của một cụm là khoảng cách giữa hai điểm xa nhau nhất trong cụm đó:

$$d(C) = \max\{\delta(x, y) \mid x, y \in C\} \quad (16)$$

  Gộp bất kỳ hai cụm nào mà sau khi gộp tạo thành một cụm mới có đường kính nhỏ nhất tại mỗi lần lặp.

Ví dụ về phân cụm phân cấp kết tụ

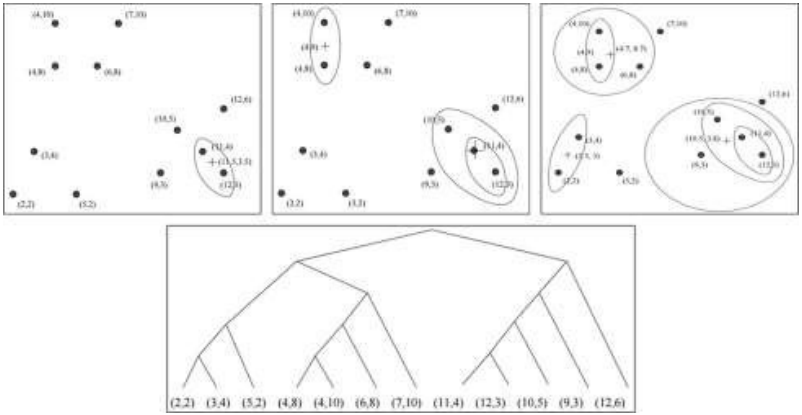


- Tập dữ liệu D bao gồm 12 điểm dữ liệu trong không gian $\mathbb{R}^2$.
- Ban đầu, mỗi điểm là một cụm riêng biệt.
- Các cặp điểm gần nhau nhất: $\delta((10,5),(11,4)) = \delta((11,4),(12,3)) = \sqrt{2}$.

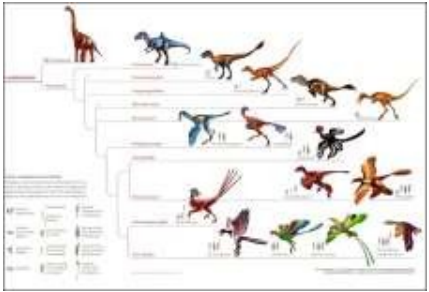
Ví dụ về phân cụm phân cấp: Quá trình gộp cụm

[(2, 2)]	[(2, 2)]	[(2, 2)]	[(2, 2)]	[(2.5, 3)]	[(2.5, 3)]	[(2.5, 3)]
[(3, 4)]	[(3, 4)]	[(3, 4)]	[(3, 4)]			
[(5, 2)]	[(5, 2)]	[(5, 2)]	[(5, 2)]	[(5, 2)]	[(5, 2)]	[(5, 2)]
[(4, 8)]	[(4, 8)]	[(4, 9)]	[(4, 9)]	[(4, 9)]	[(4.7, 8.7)]	[(4.7, 8.7)]
[(4, 10)]	[(4, 10)]					
[(6, 8)]	[(6, 8)]	[(6, 8)]	[(6, 8)]	[(6, 8)]		
[(7, 10)]	[(7, 10)]	[(7, 10)]	[(7, 10)]	[(7, 10)]	[(7, 10)]	[(7, 10)]
[(11, 4)]	[(11.5, 3.5)]	[(11.5, 3.5)]	[(11, 4)]	[(11, 4)]	[(11, 4)]	[(10.5, 3.8)]
[(12, 3)]						
[(10, 5)]	[(10, 5)]	[(10, 5)]				
[(9, 3)]	[(9, 3)]	[(9, 3)]	[(9, 3)]	[(9, 3)]	[(9, 3)]	
[(12, 6)]	[(12, 6)]	[(12, 6)]	[(12, 6)]	[(12, 6)]	[(12, 6)]	[(12, 6)]

Ví dụ về phân cụm phân cấp kết dính: kết quả



Khi nào chúng ta nên dừng gộp cụm?



- Khi chúng ta đã có kiến thức chuyên môn trước đó về số lượng cụm tiềm năng trong dữ liệu.
- Khi việc gộp cụm bắt đầu tạo ra các cụm có chất lượng thấp (ví dụ: khoảng cách trung bình từ các điểm trong một cụm đến trọng tâm của nó lớn hơn một ngưỡng cho trước).
- Khi thuật toán tạo ra toàn bộ biểu đồ cây (dendrogram), ví dụ như một cây tiến hóa.

Phân cụm phân cấp: Độ phức tạp tính toán

- Tính khoảng cách của mỗi cụm đến tất cả các cụm khác, và ở mỗi bước số lượng cụm giảm đi một. Ban đầu, mất $O(n^2)$ thời gian để tạo ma trận khoảng cách theo cặp, trừ khi nó được chỉ định là một đầu vào của thuật toán.
- Tại mỗi bước hợp nhất, khoảng cách từ cụm mới hợp nhất đến các cụm khác phải được tính lại, trong khi khoảng cách giữa các cụm còn lại vẫn giữ nguyên. Điều này có nghĩa là ở bước t, chúng ta tính $O(n - t)$ khoảng cách.
- Hoạt động chính khác là tìm cặp gần nhất trong ma trận khoảng cách. Đối với việc này, chúng ta có thể lưu n2 khoảng cách trong một cấu trúc dữ liệu heap, điều này cho phép chúng ta tìm khoảng cách nhỏ nhất trong thời gian $O(1)$; tạo heap mất thời gian $O(n^2)$.
- Xóa/cập nhật khoảng cách từ cụm hợp nhất mất $O(logn)$ thời gian cho mỗi thao tác, với tổng thời gian qua tất cả các bước hợp nhất là $O(n^2 \log n)$.
- Do đó, độ phức tạp tính toán của phân cụm theo cấp bậc là $O(n^2 \log n)$.

Nội dung

1. Khái niệm phân cụm dữ liệu
2. Hiểu dữ liệu trước khi phân cụm
3. Phân cụm phân cấp (Hierarchical clustering)
4. **Phân cụm phân vùng (Partitioningclustering)**
5. Phân cụm theo phân phối (Distribution-based)
6. Phân cụm theo mật độ (Density-based)
7. Kiểm định và đánh giá kết quả phân cụm
8. Tài liệu tham khảo và Tóm tắt

Các phương pháp phân cụm phân hoạch

- Phiên bản đơn giản và cơ bản nhất của phân tích cụm là phân hoạch, tổ chức các đối tượng của một tập hợp vào một vài nhóm hoặc cụm riêng biệt.
- Chúng ta có thể giả định rằng số lượng cụm đã được biết trước như một kiến thức nền tảng. Tham số này là điểm bắt đầu cho các phương pháp phân hoạch.
- Một cách chính quy, cho một tập dữ liệu D gồm n đối tượng và k là số cụm cần hình thành, một **thuật toán phân hoạch** tổ chức các đối tượng thành k phân hoạch ($k \leq n$), trong đó mỗi phân hoạch đại diện cho một cụm.
- Các cụm được hình thành để tối ưu hóa một **tiêu chí phân hoạch mục tiêu**, chẳng hạn như một hàm độ không tương tự dựa trên khoảng cách, sao cho các đối tượng trong cùng một cụm là "tương tự" nhau và "khác biệt" với các đối tượng ở các cụm khác xét theo các thuộc tính của tập dữ liệu.
- Các thuật toán phân hoạch phổ biến nhất là k-means, k-medoids và k-medians. Các phương pháp này sử dụng một điểm trọng tâm để đại diện cho mỗi cụm. Do đó, chúng còn được gọi là các phương pháp **đại diện** hoặc **trọng tâm**.

Xem lại bài toán phân cụm dữ liệu



- Cho $X = (X_1, X_2, \ldots, X_d)$ là một không gian d chiều, trong đó mỗi thuộc tính/biến X_j là kiểu số hoặc kiểu định danh (categorical).
- Cho $D = \{x_1, x_2, \ldots, x_n\}$ là một mẫu dữ liệu hoặc tập dữ liệu bao gồm n điểm dữ liệu (còn gọi là các thực thể, quan sát, ví dụ hoặc bộ dữ liệu) $x_i = (x_{i1}, x_{i2}, \ldots, x_{id}) \in X$.
- Phân cụm dữ liệu là sử dụng một kỹ thuật hoặc thuật toán phân cụm A để gán các điểm dữ liệu trong D vào các cụm có khả năng nhất của chúng. Kết quả phân cụm là một tập gồm k cụm $C = \{C_1, C_2, \ldots, C_k\}$.
- Các điểm dữ liệu trong cùng một cụm thì tương tự nhau theo một nghĩa nào đó và cách xa các điểm dữ liệu trong các cụm khác.

Thuật toán K-means

- Cho $C = \{C_1, C_2, \ldots, C_k\}$ là một giải pháp phân cụm, chúng ta cần một hàm số điểm để đánh giá chất lượng của nó trên D. Hàm số điểm tổng bình phương sai số (SSE) này được định nghĩa là:

$$SSE(C, D) = \sum_{i=1}^{k} \sum_{x_j \in C_i} \|x_j - \mu_i\|^2 \quad (17) \tag{1}$$

- Mục tiêu là tìm giải pháp phân cụm C^* tối thiểu hóa điểm SSE:

$$C^* = \arg\min_C SSE(C, D) \quad (18)$$

$$= \arg\min_C \left(\sum_{i=1}^{k} \sum_{x_j \in C_i} \|x_j - \mu_i\|^2 \right) \tag{2}$$

- Thuật toán k-means sử dụng phương pháp **lặp tham lam** để tìm một giải pháp phân cụm tối thiểu hóa mục tiêu SSE. Do đó, nó có thể hội tụ về một cực trị địa phương thay vì cụm tối ưu toàn cục.

Thuật toán K-means (tiếp theo)

- k-means khởi tạo các giá trị trung bình của cụm bằng cách tạo ngẫu nhiên k điểm trong không gian dữ liệu. Điều này thường được thực hiện bằng cách tạo một giá trị ngẫu nhiên đồng nhất trong phạm vi của mỗi chiều.
- Mỗi lần lặp của k-means bao gồm hai bước:
  - Gán cụm, và
  - Cập nhật trọng tâm hoặc giá trị trung bình.
- Cho k giá trị trung bình của cụm, trong **bước gán cụm**, mỗi điểm $x_j \in D$ được gán cho giá trị trung bình gần nhất, tạo ra một phân cụm với mỗi cụm C_i bao gồm các điểm gần với μ_i hơn bất kỳ giá trị trung bình cụm nào khác. Nghĩa là, mỗi điểm x_j được gán vào cụm C_{j^*}, trong đó:

$$j^* = \arg\min_{i=1..k} \|x_j - \mu_i\|^2 \quad (19) \qquad (3)$$

Thuật toán K-means (tiếp theo)

- Cho một tập các cụm $C_i, i = 1..k$, trong **bước cập nhật trọng tâm**, các giá trị trung bình mới được tính toán cho mỗi cụm từ các điểm trong C_i.
- Các bước gán cụm và cập nhật trọng tâm được thực hiện lặp đi lặp lại cho đến khi chúng ta đạt được một điểm cố định hoặc cực tiểu địa phương.
- Về mặt thực tế, người ta có thể coi k-means đã hội tụ nếu các trọng tâm không thay đổi từ lần lặp này sang lần lặp tiếp theo. Ví dụ, chúng ta có thể dừng nếu:

$$\sum_{i=1}^{k} \|\mu_i^t - \mu_i^{t-1}\|^2 \leq \epsilon \quad (20) \qquad (4)$$

trong đó $\epsilon > 0$ là ngưỡng hội tụ, và t biểu thị lần lặp hiện tại.

Thuật toán K-means: Mã giả

K-MEANS (D, k, ϵ):

1. $t = 0$
2. Khởi tạo ngẫu nhiên k trọng tâm: $\mu_1^t, \mu_2^t, \ldots, \mu_k^t \in \mathbb{R}^d$
3. **repeat**
4. $\quad t \leftarrow t + 1$ // Bước gán cụm (Cluster Assignment Step)
5. $\quad$ **foreach** $x_j \in D$ **do**
6. $\quad\quad j^* \leftarrow \arg\min_i \{\|x_j - \mu_i^t\|^2\}$ // Gán x_j vào trọng tâm gần nhất
7. $\quad\quad C_{j^*} \leftarrow C_{j^*} \cup \{x_j\}$ // Bước cập nhật trọng tâm (Centroid Update Step)
8. $\quad$ **foreach** $i = 1$ đến k **do**
9. $\quad\quad \mu_i^t \leftarrow \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j$
10. **until** $\sum_{i=1}^{k} \|\mu_i^t - \mu_i^{t-1}\| \leq \epsilon$

Thuật toán K-means: Độ phức tạp tính toán

- **Bước gán cụm** tốn thời gian $O(nkd)$, vì với mỗi điểm trong số n điểm, chúng ta phải tính khoảng cách của nó tới mỗi cụm trong số k cụm, mỗi phép tính tốn d thao tác trong không gian d chiều.
- **Bước tính toán lại trọng tâm** tốn thời gian $O(nd)$, vì chúng ta phải cộng tổng của n điểm dữ liệu d chiều.
- Giả sử có t lần lặp, **tổng thời gian** cho K-means là $O(tnkd)$.
- Về **chi phí I/O**, thuật toán yêu cầu $O(t)$ lần quét toàn bộ cơ sở dữ liệu, vì chúng ta phải đọc toàn bộ dữ liệu trong mỗi lần lặp.

Thuật toán K–means: ví dụ 1

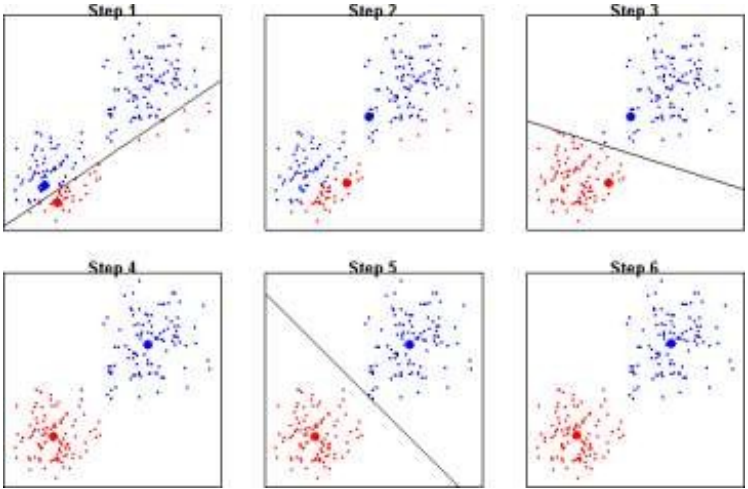


Figure: Phân cụm với K-means [nguồn: sherrytowers.com/2013/10/24/k-means-clustering]

Thuật toán K–means: ví dụ 2

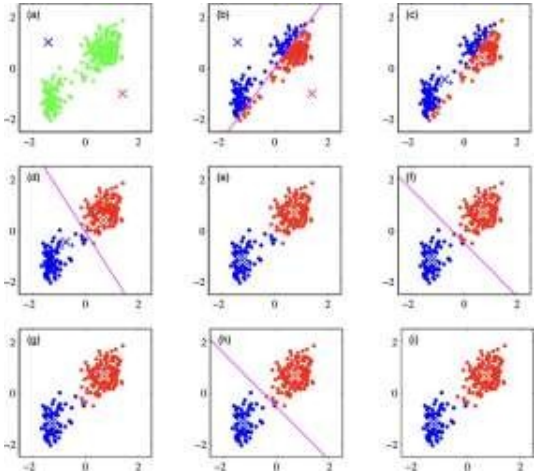


Figure: Phân cụm với K-means [trích từ Pattern Recognition and Machine Learning của C.M. Bishop]

Thuật toán K–means: ví dụ 3 (phân đoạn ảnh)



Figure: Phân đoạn ảnh với k–means [trích từ Pattern Recognition and Machine Learning của C.M. Bishop]

Khởi tạo cho k vector trung bình μ_i

- Các giá trị trung bình ban đầu nên nằm ở các cụm khác nhau. Có hai cách tiếp cận:
  - Chọn các điểm nằm càng xa nhau càng tốt.
  - Phân cụm một mẫu (nhỏ) dữ liệu, có lẽ theo phương pháp phân cấp, để có k cụm. Chọn một điểm từ mỗi cụm, có thể là điểm gần trọng tâm của cụm đó nhất.
- Cách tiếp cận thứ hai yêu cầu chuẩn bị kỹ lưỡng hơn. Đối với cách tiếp cận thứ nhất, có một lựa chọn tốt là:

Thuật toán chọn điểm xa nhau

Chọn điểm đầu tiên ngẫu nhiên;
WHILE có ít hơn k điểm **DO**
  Thêm điểm có khoảng cách tối thiểu đến các điểm đã chọn là lớn nhất có thể;
END

Ví dụ về khởi tạo k vector trung bình μ_i

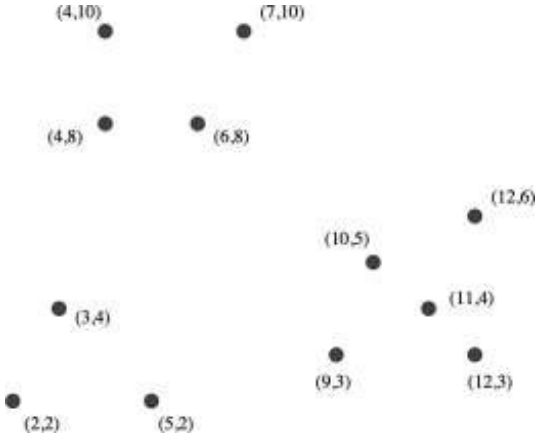


Figure: Lựa chọn ban đầu cho các giá trị trung bình [trích từ Mining of Massive Datasets của J. Leskovec và cộng sự]

Ví dụ về khởi tạo k vector trung bình μ_i (tiếp theo)

- Trong trường hợp xấu nhất, lựa chọn ban đầu là một điểm gần trung tâm, ví dụ $(6, 8)$. Điểm xa nhất từ $(6, 8)$ là $(12, 3)$, vì vậy điểm đó được chọn tiếp theo.
- Trong mười điểm còn lại, điểm có khoảng cách tối thiểu đến $(6, 8)$ hoặc $(12, 3)$ đạt giá trị lớn nhất là $(2, 2)$. Điểm đó có khoảng cách $\sqrt{52} = 7.21$ từ $(6, 8)$ và khoảng cách $\sqrt{101} = 10.05$ đến $(12, 3)$; do đó "điểm số" của nó là 7.21.
- Chúng ta có thể kiểm tra dễ dàng rằng bất kỳ điểm nào khác đều có khoảng cách nhỏ hơn 7.21 đến ít nhất một trong hai điểm $(6, 8)$ và $(12, 3)$. Lưu ý rằng ba điểm này thuộc về các cụm khác nhau.
- Nếu chúng ta bắt đầu với một điểm khác, ví dụ $(10, 5)$, chúng ta sẽ nhận được một tập hợp ba điểm khởi đầu khác là $(10, 5)$, $(2, 2)$ và $(4, 10)$. Một lần nữa, các điểm này thuộc về ba cụm khác nhau.

K-means nhạy cảm với các điểm ngoại lai

- Thuật toán k-means **nhạy cảm với các điểm ngoại lai** vì các đối tượng này nằm xa đa số dữ liệu, và do đó, khi được gán vào một cụm, chúng có thể làm biến dạng đáng kể giá trị trung bình của cụm đó.
- Điều này vô tình ảnh hưởng đến việc gán các đối tượng khác vào cụm. Hiệu ứng này càng trở nên nghiêm trọng hơn do việc sử dụng bình phương sai số.
- **Ví dụ:** xét 7 điểm dữ liệu trong không gian 1 chiều: $1, 2, 3, 8, 9, 10, 25$, với $k = 2$.
  - Trực quan qua quan sát, chúng ta có thể hình dung các điểm được phân chia thành các cụm $\{1, 2, 3\}$ và $\{8, 9, 10\}$, trong đó điểm 25 bị loại bỏ vì nó có vẻ là một điểm ngoại lai.
  - **Giải pháp 1:** $\{1, 2, 3\}$ với trung bình $= 2$ và $\{8, 9, 10, 25\}$ với trung bình $= 13$. Sai số là: $(1-2)^2 + (2-2)^2 + (3-2)^2 + \cdots + (10-13)^2 + (25-13)^2 = 196$
  - **Giải pháp 2:** $\{1, 2, 3, 8\}$ với trung bình $= 3.5$ và $\{9, 10, 25\}$ với trung bình $= 14.67$. Sai số là:
    $(1-3.5)^2 + (2-3.5)^2 + (3-3.5)^2 + (10-14.67)^2 + (25-14.67)^2 = 189.67$
  - **Giải pháp 2** được chọn vì có tổng bình phương sai số thấp nhất. Tuy nhiên, điểm 8 lẽ ra không nên ở cụm 1. Ngoài ra, trung bình của cụm thứ hai là 14.67, khá xa so với 9 và 10 do ảnh hưởng của điểm ngoại lai 25.

Thuật toán phân cụm K-medoids

- Thay vì sử dụng các giá trị trung bình, k-medoids chọn các đối tượng dữ liệu thực tế trong tập dữ liệu để đại diện cho các cụm, sử dụng một đối tượng đại diện cho mỗi cụm.
- Mỗi đối tượng còn lại được gán vào cụm có đối tượng đại diện tương tự nhất với nó.
- Phương pháp phân hoạch sau đó được thực hiện dựa trên nguyên lý tối thiểu hóa tổng độ không tương tự giữa mỗi đối tượng x và đối tượng đại diện tương ứng o_i của nó. Nghĩa là, một **tiêu chí sai số tuyệt đối** được sử dụng, định nghĩa là:

$$E = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - o_i\|_1 \quad (21) \qquad (5)$$

- Đây là cơ sở cho phương pháp k-medoids, giúp nhóm n đối tượng thành k cụm bằng cách tối thiểu hóa sai số tuyệt đối.
- Khi $k = 1$, chúng ta có thể tìm thấy trung vị chính xác trong thời gian $O(n^2)$. Tuy nhiên, khi k là một số nguyên dương tổng quát, bài toán k-medoids là NP-khó.

Thuật toán K-medoids: Phân hoạch quanh các Medoid (PAM)

- Thuật toán phân hoạch quanh các medoid (PAM) là một hiện thực hóa phổ biến của phân cụm k-medoids. Nó giải quyết vấn đề theo cách **lặp đi lặp lại và tham lam**.
- Giống như thuật toán k-means, các đối tượng đại diện ban đầu (gọi là các hạt giống - seeds) được chọn một cách tùy ý.
- Chúng ta xem xét liệu việc thay thế một đối tượng đại diện bằng một đối tượng không đại diện có cải thiện chất lượng phân cụm hay không. Tất cả các thay thế khả thi đều được thử nghiệm.
- Quá trình lặp đi lặp lại việc thay thế các đối tượng đại diện bằng các đối tượng khác tiếp tục cho đến khi chất lượng của phân cụm kết quả không thể cải thiện thêm bởi bất kỳ sự thay thế nào.
- Chất lượng này được đo bằng một hàm chi phí là tổng độ không tương tự giữa mọi đối tượng dữ liệu và đối tượng đại diện của cụm tương ứng.

Thuật toán K-medoids: Phân hoạch quanh các Medoid (PAM) (tiếp theo)

- Cụ thể, gọi $o_1, o_2, \ldots, o_k$ là tập hợp hiện tại của các đối tượng đại diện (medoids) của k cụm.
- Để xác định xem một đối tượng không đại diện, ký hiệu là o_{random}, có phải là sự thay thế tốt cho medoid hiện tại o_j $(1 \leq j \leq k)$ hay không, chúng ta tính toán khoảng cách từ mọi đối tượng x đến đối tượng gần nhất trong tập $\{o_1, \ldots, o_{j-1}, o_{random}, o_{j+1}, \ldots, o_k\}$, và sử dụng khoảng cách đó để cập nhật hàm chi phí.
- Việc tái gán các đối tượng rất đơn giản:
  - Giả sử một đối tượng x hiện đang được gán cho cụm đại diện bởi medoid o_j: x cần được gán lại cho o_{random} hoặc một cụm khác đại diện bởi o_i $(i \neq j)$, tùy theo đối tượng nào gần nhất.
  - Giả sử một đối tượng x hiện đang được gán cho cụm đại diện bởi một o_i khác $(i \neq j)$: x vẫn được gán cho o_i miễn là x vẫn gần o_i hơn so với o_{random}. Ngược lại, x được gán lại cho o_{random}.
- Nếu sai số E giảm xuống, hãy thay thế o_j bằng o_{random}. Ngược lại, o_j được chấp nhận và không có gì thay đổi trong lần lặp đó. Thuật toán sẽ dừng khi không có sự thay đổi nào về sai số E với tất cả các thay thế khả thi.

Phương pháp nào bền vững hơn? k-means hay k-medoids?

- Phương pháp k-medoids **bền vững hơn** k-means khi có sự hiện diện của nhiễu và các điểm ngoại lai vì một medoid ít bị ảnh hưởng bởi các điểm ngoại lai hoặc các giá trị cực đoan khác hơn so với giá trị trung bình.
- Tuy nhiên, độ phức tạp của mỗi lần lặp trong thuật toán k-medoids là $O(k(n-k)^2)$.
- Đối với các giá trị n và k lớn, việc tính toán như vậy trở nên **rất tốn kém**, và tốn kém hơn nhiều so với phương pháp k-means.
- Cả hai phương pháp đều yêu cầu người dùng chỉ định k, số lượng cụm.
- Một thuật toán phân hoạch k-medoids điển hình như PAM hoạt động hiệu quả cho các tập dữ liệu nhỏ, nhưng không mở rộng tốt cho các tập dữ liệu lớn. Làm thế nào để mở rộng phương pháp k-medoids? Để xử lý các tập dữ liệu lớn hơn, một phương pháp dựa trên lấy mẫu có tên là **CLARA** (Clustering LARge Applications) có thể được sử dụng.

Thuật toán phân cụm k-medians

- Trong thuật toán k-medians, khoảng cách Manhattan (khoảng cách L_1) được sử dụng trong hàm mục tiêu thay vì khoảng cách Euclidean (khoảng cách L_2). Hàm mục tiêu trong k-medians là:

$$E = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - m_i\|_1 = \sum_{i=1}^{k} \sum_{x \in C_i} \sum_{j=1}^{d} |x_j - m_j| \quad (22) \qquad (6)$$

  trong đó m_i là trung vị của các điểm dữ liệu theo từng chiều trong cụm C_i. Điều này là do điểm có tổng khoảng cách L_1 nhỏ nhất đến một tập hợp các điểm phân bố trên một đường thẳng chính là trung vị của tập hợp đó.
- Vì trung vị được chọn độc lập theo từng chiều, nên điểm đại diện d chiều kết quả (thông thường) sẽ không thuộc tập dữ liệu gốc D. Cách tiếp cận k-medians đôi khi bị nhầm lẫn với cách tiếp cận k-medoids, vốn chọn các điểm đại diện từ chính tập dữ liệu gốc D.
- Cách tiếp cận k-medians thường chọn các điểm đại diện cụm theo cách bền vững hơn k-means, bởi vì trung vị không nhạy cảm với sự hiện diện của các điểm ngoại lai trong cụm như giá trị trung bình.

Nội dung

1. Khái niệm phân cụm dữ liệu
2. Hiểu dữ liệu trước khi phân cụm
3. Phân cụm phân cấp (Hierarchical clustering)
4. Phân cụm phân vùng (Partitioning clustering)
5. **Phân cụm theo phân phối (Distribution-based)**
6. Phân cụm theo mật độ (Density-based)
7. Kiểm định và đánh giá kết quả phân cụm
8. Tài liệu tham khảo và Tóm tắt

Tài liệu tham khảo

1. J. Han, M. Kamber, và J. Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, Elsevier, 2012.
2. C. Aggarwal. *Data Mining: The Textbook*. Springer, 2015.
3. J. Leskovec, A. Rajaraman, và J. D. Ullman. *Mining of Massive Datasets*. Cambridge University Press, 2014.
4. M. J. Zaki và W. M. Jr. *Data Mining and Analysis: Fundamental Concepts and Algorithms*. Cambridge University Press, 2013.
5. D. Easley và J. Kleinberg. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010.
6. J. VanderPlas. *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly, 2017.
7. J. Grus. *Data Science from Scratch: First Principles with Python*. O'Reilly, 2015.

Tóm tắt

- Giới thiệu các khái niệm quan trọng về phân cụm: định nghĩa, các loại phân cụm (cứng và mềm), các yêu cầu chính, các phương pháp tiếp cận, thách thức và ứng dụng của phân cụm.
- Các kỹ thuật chính để hiểu phân phối dữ liệu trước khi phân cụm: biểu đồ không gian (spatial histogram), entropy dựa trên ô (cell-based entropy), phân phối khoảng cách và thống kê Hopkins.
- Phương pháp phân cụm phân cấp với thuật toán kết tụ (bottom-up), biểu đồ cây (dendrogram), các cách khác nhau để gộp cụm (liên kết đơn, liên kết đầy đủ, liên kết trung bình, liên kết trọng tâm, bán kính và đường kính).
- Phương pháp phân cụm phân hoạch với thuật toán k-means, khởi tạo k trọng tâm, và các biến thể của k-means bao gồm k-medoids (thuật toán PAM) và k-medians.